

A PRELIMINARY SKETCH OF
A POLYMORPHIC PROGRAMMING LANGUAGE

ROUGH DRAFT

Tim Standish
Centro de Cálculo Electronico
U.N.A.M.
June 24, 1968

A B S T R A C T

This paper, written in a telegraphic and sloppy style, attempts to give a rough intuitive impression of the Polymorphic Programming Language, principally by means of simple examples. Formal definitions of the properties of the language are not furnished.

INTRODUCTION

The Polymorphic Programming Language is an extensible conversational programming language based on Iverson's APL [1,2] , the Brooker and Morris Compiler-Compiler [3,4,9] , and a Data Definition Facility of the author [5] .

A base language is supplied in which one may define extensions to the syntax and semantics of expressions. The base language is an operator precedence language somewhat similar in syntax to Iverson's APL and extended to incorporate data definition and syntax extension mechanisms. A variant of the lambda calculus of Church [7] is employed for operator definitions.

One may define data spaces, ~~classes of~~ operations on the data defined and notations evoking these operations. In this fashion, language extensions for such tasks as formula, list, string and file processing may be created. In addition, new types of arithmetics may be added such as polynomial, rational, matrix and complex arithmetic. If desired, all types of arithmetic defined in the system may share one common arithmetic notation.

Defined notations may be saved as collections of definitions on personal files or in common libraries. A programmer may adapt or ^ccreate a notation to suit his personal tastes or the requirements of a given task. He may modify a given notation progressively in response to his changing needs during a series of programming sessions.

In addition, ~~one property of~~ PPL permits notation to be dynamic and to hold for a scope specified either conversationally ~~by the programmer~~ or by a program. Thus, which subset of defined notations is in effect at a given moment is subject to programmer control. In particular, a given syntax may be used at two different times with different meanings. Thus, a given notation may be shared dynamically amongst several meanings at different stages of a conversation. Furthermore, by revoking notations not in use for a certain scope, the efficiency of the system can be improved.

It is also possible in PPL to demonstrate a kind of polymorphic mechanics which arises when, having many forms of data in the system at once, we write programs that are both independent of data form and applicable to all data forms. We will find it possible, in certain cases, to factor out the mechanics of data sequencing common to many processes and to express this sequencing in one abstract process. It

will be possible, in these cases, to write data independent algorithms in compact forms that are free from contamination by the mechanics of data sequencing.

THE BASE LANGUAGE

The Base Language has a simple grammar and some simple parsing rules. The grammar G of the base language is as follows:

$$\begin{aligned} E &\rightarrow T\beta E \mid \alpha E \mid T \\ T &\rightarrow T(L) \mid T[L] \mid P \\ P &\rightarrow (L) \mid [L] \mid \lambda \\ L &\rightarrow E \mid E, L \end{aligned}$$

In this grammar α , β and λ are lexemes of the language recognized by a lexical analyzer. These lexemes include the following:

$$\alpha = (+, -, \uparrow, \div, \uparrow, \downarrow, \circ, \sim, ?, *, \rightarrow, \nabla, \Delta)$$

$$\beta = (+, -, x, +, \uparrow, \uparrow, \downarrow, \downarrow, /, \wedge, \vee, \rightarrow, \leftarrow, =, \neq, >, <, \geq, \leq, :, *, ==)$$

$$\lambda = \text{identifier, number, logical value, string, type, } \square, \\ \text{extractor } (\underline{X}), \text{ region choice } (\bar{X}), \text{ underlined operator } (\underline{e})$$

(NOTE: Some of the unary and binary operators are identical in appearance. For example : $+, -, \uparrow, \div, \lceil, \lfloor, \rightarrow, \circ, *$ are both unary and binary. The lexical analyzer must classify them therefore on the basis of their context. The grammar G is carefully constructed so that binary operators are preceded only by the characters $\{),], \lambda\}$, and unary operators are never preceded by $\{),], \lambda\}$ but instead by $\{?, [, (, \uparrow, \alpha, \beta\}$. Hence, by a cheap test on the preceding character, the lexical analyzer can distinguish between unary and binary operators.)

The grammar G is both a phrase structure grammar and an operator precedence grammar. Appendix I gives the precedence matrix, precedence functions , and the algorithm for recognizing expressions in precedence grammars, following procedures given by Floyd [8].

Some examples of expressions in G are as follows:

$A[2,3] \leftarrow P12 + F(X, G(2,T))$
 $I \times I + J \times J$
 $PAIR(X) = [HEAD: X, TAIL: LIST(X)]$
 $[SAE] = [? PM][T][? * PMT]$
 $F(G,2)(E \wedge \sim T, V)$
 $\sqrt{HEAD(L)[2,4] \leftarrow X}$
 $F \leftarrow \nabla(N) ((N=0) \rightarrow (1, N \times F(N-1)))$
 $I = \nabla(X,Y)(matrixadd(X,Y))$

There are some simple rules for parsing expressions in G, the most important of which is that Iversonian* precedence is used to parse all expressions involving unary or binary operators. Iversonian precedence is different from normal arithmetic precedence. In particular, the following rules hold for parsing expressions in the PPL base language:

1. Constants, identifiers, operators and the punctuation marks { (,), [,], , } are parsed immediately.
2. Any phrase $\emptyset$ inside parentheses ($\emptyset$) or brackets [$\emptyset$] is parsed before any expression containing the phrases ($\emptyset$) or [$\emptyset$] .
3. Juxtaposed phrases $\emptyset$ of the form E(E) or E[E] are parsed with association to the left before any phrase containing $\emptyset$.
4. All other phrases are parsed by the rule of Iversonian precedence which states that every operator takes as its right hand argument the entire expression to its right up to an enclosing parenthesis) , right bracket] , comma , , or end of line.

Examples:

HEAD(X)[1,3] is parsed (HEAD(X))[1,3] not HEAD((X) [1,3])
according to rule 3.

I↑2 + SIN(X) is parsed I↑(2+SIN(X)) not (I↑2)+ SIN(X)
according to rule 4.

Not all strings in the base language have defined values.

That is, some expressions have the special value undefined.

*Perhaps it is a misnomer to call this Iversonian precedence for this form of precedence was used in the GATE and IT compilers written by Perlis, Van Zoeren et al at Carnegie Tech in the late fifties.

However, at any time, a programmer may supply semantics for any such undefined expression by making appropriate definitions.

THE FORMAT OF DEFINITIONS

Definitions of new functions, new syntax and new data formats are preceded by the define operator Δ .

An example of a function definition is:

```

 $\Delta$  F =  $\nabla$ (N)
1: LOCAL(S,I)
2: S  $\leftarrow$  0
3: I  $\leftarrow$  1
4: (I  $\triangleright$  N)  $\rightarrow$  ( $\rightarrow$  8)
5: S  $\leftarrow$  S + I  $\uparrow$  P
6: I  $\leftarrow$  I+1
7:  $\rightarrow$  4
8: F  $\leftarrow$  S
9: .

```

Here, F is a function of the variable N. In step 1, S and I are made variables local to the function. When the syntax of a definition detects a function definition operator, ∇ ,

line numbers are furnished unless the definition is concluded with a period on the first line. One is permitted to write one expression in G per numbered line when writing in the base language. Step 4 in the above function is an example of a conditional. The statement $(I > N) \rightarrow (\rightarrow 8)$ is the equivalent of the ALGOL statement: if $I > N$ then go to 8 . In step 7, the statement $\rightarrow 4$ is the equivalent of the ALGOL : go to 4. Thus the right arrow $\rightarrow$ is a conditional operator when it is dyadic and is a go to operator when it is monadic. Two arm conditionals such as 'if B then C else D' are written $B \rightarrow (C, D)$ in PPL. This notation follows CPL [6].

An example of a function definition on one line is the factorial function:

$$\Delta F = \nabla(N) (F \leftarrow (N=0) \rightarrow (1, N \times F(N-1))) .$$

This function is recursive.

An example of a data definition is the following:

$$\Delta \text{COMPLEX} = [\text{REALPART:}\underline{\text{REAL}}, \text{IMAGPART:}\underline{\text{REAL}}] .$$

This declares a COMPLEX to be a compound data object having

two parts: a REALPART which is of type REAL and an IMAGPART which is of type REAL. Once this data definition has been made there arise automatically several functions which may be used in the writing of operations on COMPLEXes. For example, the functions REALPART(X) and IMAGPART(X) may be applied to any X which is a COMPLEX. These functions are decomposition functions or selector functions whose purpose is to access the parts of a compound data format representing a COMPLEX. Selector functions may also be used on the left hand side of assignment statements, as in REALPART(X) ← 5, to change components of compound data structures. Assuming that X is COMPLEX, the assignment REALPART(X) ← 5 changes the REALPART of X to be equal to 5. Another class of functions which are created as a result of defining COMPLEXes are the constructor functions COMPLEX◦(A,B) and ◦COMPLEX. Provided that A and B are of type REAL, the value of the expression COMPLEX◦(A,B) is a COMPLEX with REALPART A and IMAGPART B. The value of the expression ◦COMPLEX is a COMPLEX whose REALPART and IMAGPART have the value undefined. In this case, the REALPART and IMAGPART must be defined by assignment. Finally, if X is a COMPLEX, then the value of the expression TYPE(X) is COMPLEX. Each constructed datum has a type which is a type expression. The word COMPLEX is a type expression as are the words REAL, INTEGER and BOOLEAN. In general, both

the name of the constructor function and the type expression of constructed data are formed from the expression found on the left hand side of the equal sign in a data definition.

Thus , in the definition:

$$\Delta \text{ COMPLEX} = [\text{REALPART:}\underline{\text{REAL}}, \text{IMAGPART: } \underline{\text{REAL}}] .$$

the word COMPLEX is a type expression. Two COMPLEXes A and B may be tested for equality by the expression A=B. This will be the case if and only if REALPART(A)=REALPART(B) and IMAGPART(A)=IMAGPART(B). One can test an arbitrary data object to see if it is COMPLEX by means of the predicate X == COMPLEX. In this case the expression X == COMPLEX is true if and only if TYPE(X)= COMPLEX.

An example of a function that could be written after making a data definition of COMPLEX is the following function for finding the sum of two complex numbers:

$$\Delta \text{ ComplexAdd} = \nabla (X,Y)$$

1: ComplexAdd $\leftarrow$ COMPLEX $\circ$ (REALPART(X)+REALPART(Y), ...

1: IMAGPART(X)+IMAGPART(Y)).

The dierisis "... " is a continuation character which permits the next line to be considered part of the line terminated by "...".

Another example of a data definition is the following series of definitions for Lists of Objects of Type X:

$$\Delta \text{LIST}(X) = \text{NULLIST} \vee \text{PAIR}(X)$$
$$\Delta \text{PAIR}(X) = [\text{HEAD}:X, \text{TAIL}:\text{LIST}(X)]$$
$$\Delta \text{NULLIST} = \text{NIL}$$

In these definitions, the occurrences of X's are occurrences of formal parameters. Any type expression can be substituted for X throughout these definitions in order to create definitions of Lists whose elements are of the type specified by the substituted type expression. For example, to create definitions of Lists of Integers, the use of the expression LIST(INTEGER) would cause the substitution of the type expression INTEGER for X everywhere in the above definition, resulting in the following set of definitions:

$$\Delta \text{LIST}(\text{INTEGER}) = \text{NULLIST} \vee \text{PAIR}(\text{INTEGER})$$
$$\Delta \text{PAIR}(\text{INTEGER}) = [\text{HEAD}:\text{INTEGER}, \text{TAIL}:\text{LIST}(\text{INTEGER})]$$
$$\Delta \text{NULLIST} = \text{NIL}$$

As before, there arise automatically from the creation of such a definition, classes of functions on lists including selectors, constructors and predicates. In particular, the following arise. If P is a PAIR(INTEGER), then HEAD(P)

and TAIL(P) are selectors whose values are in the domains of INTEGERS and LIST's of INTEGER's, respectively. PAIR(INTEGER) $\circ$ (I,L) constructs from an integer I and a LIST of INTEGER's L, a PAIR(INTEGER) which is a compound data format whose HEAD is I and whose TAIL is L. If L is a LIST(INTEGER), then TYPE(L) = PAIR(INTEGER) or TYPE(L) = NULLLIST. Finally, the predicate $X = \text{LIST}(\text{INTEGER})$ is true if and only if either TYPE(L) = NULLLIST or TYPE(L) = PAIR(INTEGER).

An example of a routine to find the length (or number of elements) in the list L is as follows:

$\Delta \text{LENGTH} = \nabla (L)$

$1:\text{LENGTH} \leftarrow (L = \text{NULLLIST}) \rightarrow (0, 1 + \text{LENGTH}(\text{TAIL}(L)))$.

A list of elements of arbitrary type is specified by the value of the expression LIST(GENERAL). Thus, we see that the definition of LIST(X) with formal parameter X is, in fact, a definition schema for manufacturing an indefinite number of definitions of lists with particular types of elements.

There are several other types of expressions that can be used in data definitions. We will explain them as we encounter them later.

An example of some syntax definition formats is as follows:

$$\Delta [\text{SAE}] = [?\text{+-}][\text{T}][?* \text{+-T}]$$

$$\Delta [*X] = [X][*X], [X]$$

$$\Delta [?X] = [X], \text{NIL}$$

The construction $[*X]$ means a sequence of one or more occurrences of the phrase class designated by $[X]$. In the second definition above, we see that this means $[*X]$ has one of two possible formats: either $[X][*X]$ meaning an occurrence of $[X]$ followed by one or more occurrences of $[X]$, or a single occurrence of $[X]$. The comma means or in these definitions. The construction $[?X]$ means either the presence or the absence of an $[X]$. The third definition defines $[?X]$ as consisting of one of two formats: either an $[X]$ or a NIL denoting the absence of an $[X]$. In the first definition, the phrase class $[\text{SAE}]$ is defined to consist of an optional occurrence of the class $[\text{+-}]$ followed by an occurrence of the class $[\text{T}]$ followed by an optional occurrence of one or more occurrences of the class $[\text{+-T}]$. These usages follow Brooker and Morris [3,4,9] with only minor variation. In particular, though the operators $*$ and $?$ have the same meaning as in the Brooker and Morris system, they are used here as unary prefix operators instead of as unary postfix operators as in the Compiler-Compiler.

Other expressions for defining syntax are possible in PPL and will be both used and explained in the sequel.

SYNTAX EXTENSIONS IN PPL

In this paragraph we shall explain and exemplify four methods of extending the syntax of the base language in PPL. These are:

- (1) Operator Definitions,
- (2) Revision of Operator Definitions,
- (3) Syntax Macros , and
- (4) Brooker and Morris Style Compilation.

OPERATOR DEFINITIONS

New monadic or dyadic operators may be added to the base language. New operators added follow the Iversonian precedence rules. It is therefore unnecessary to define their precedence explicitly. What is necessary is to define the semantics of the operator when used as a monadic or dyadic operator or both.

For example to introduce the $\lceil$ operator as a dyadic operator meaning maximum, the following definition would be made:

$$\Delta \lceil = \nabla (X,Y)$$

$$1: \lceil \leftarrow (X > Y) \rightarrow (X,Y) .$$

If it is desired to introduce the $\lceil$ operator as a unary operator meaning "ceiling" at the same time that $A \lceil B$ means maximum, this can be done by the following definition:

$\Delta \lceil = \nabla(X)$
 1: LOCAL(Y)
 2: $Y \leftarrow \text{RESIDUE}(X, 1)$
 3: $\lceil \leftarrow (Y=0) \rightarrow (X, 1+X-Y)$

The system is smart enough to know that when an operator is defined with two arguments it is the definition to be used for the dyadic occurrences of the operator, and when it is defined with one argument it is the definition to be used for the monadic occurrences of the operator. The monadic and dyadic definitions may hold simultaneously and may be different.

Any word spelled from operators θ , underlined and used in the context $\Delta \underline{\theta} = \nabla(\dots)$ will be recognized as a single unit after the definition has been accepted by the system. This permits the definition of compound operators such as $\{=, ::, \text{ and } *\#\%$. In expressions of the grammar G, when operators are used in place of occurrences of λ , they must be underlined. Underlined operators are recognized by the lexical analyzer as instances of λ 's. For example, operators used on the left hand side of assignments, as function designators and as arguments in lists must be underlined.

REVISION OF OPERATOR DEFINITIONS

To revise or extend the meaning of operators already defined in the system one edits the text of the definition supplied in the system, (and recompiles if desired).

For example, if we are supplied with integer, real and matrix arithmetic at a given stage of extension of the base language and we want to add complex and rational arithmetic, we might need to expand the definition of '+' ..

If the current definition of binary '+' is:

$\Delta \underline{+} = \nabla (X,Y)$

1: SWITCH ON [type(X), type(Y)] INTO ...

1: begin....

1: [REAL,REAL] :: $\underline{+} \leftarrow \text{realadd}(X,Y)...$

1: [INTEGER,REAL] :: $\underline{+} \leftarrow \text{realadd}(\text{integertoreal}(X),Y)...$

1: [REAL,INTEGER] :: $\underline{+} \leftarrow \text{realadd}(X,\text{integertoreal}(Y))...$

1: [INTEGER,INTEGER] :: $\underline{+} \leftarrow \text{integeradd}(X,Y)...$

1: [MATRIX,MATRIX] :: $\underline{+} \leftarrow \text{matrixadd}(X,Y)...$

1: ALLOTHERS :: $\underline{+} \leftarrow \text{"ADDITION ERROR" } ...$

1: end.

Then we could expand this definition by adding the cases

[COMPLEX,COMPLEX] :: $\underline{+} \leftarrow \text{complexadd}(X,Y)...$

[RATIONAL,RATIONAL] :: $\underline{+} \leftarrow \text{rationaladd}(X,Y)...$

where we had defined as data:

Δ COMPLEX = [REALPART:REAL, IMAGPART:REAL]

Δ RATIONAL = [NUMERATOR:INTEGER, DENOMINATOR:INTEGER]

and where we had defined as operations:

Δ complexadd = $\nabla(X,Y)$

1: complexadd $\leftarrow$ COMPLEX $\circ$ (REALPART(X)+REALPART(Y),...

1: IMAGPART(X)+IMAGPART(Y)) .

and

Δ rationaladd = $\nabla(X,Y)$

1: LOCAL(N,D,G)

2: N $\leftarrow$ ((NUMERATOR(X)*DENOMINATOR(Y))+(NUMERATOR(Y)*DENOMINATOR(X)

3: D $\leftarrow$ DENOMINATOR(X) * DENOMINATOR(Y)

4: G $\leftarrow$ gcd(N,D)

5: rationaladd $\leftarrow$ RATIONAL $\circ$ (N+G, D+G)

6: .

Note: some primitive functions such as realadd(x,y) , integeradd(x,y) and integertoreal(x,y) are supplied in the base language. These are analagous to car,cdr,cons,atom,eq and null in LISP 1.5 [11] .

SYNTAX MACRO EXTENSIONS OF SYNTAX

Consider the following syntax extension definition:

```
 $\Delta[FS] =$  FOR  $\underline{I}$ :  $[E] \leftarrow \underline{L}$ :  $[E]$  TO  $\underline{U}$ :  $[E]$  DO  $\underline{S}$ :  $[E]$   
1:  $\underline{I} \leftarrow \underline{L}$   
2:  $(\underline{I} > \underline{U}) \rightarrow (\rightarrow \underline{NEXT})$   
3:  $\underline{S}$   
4:  $\underline{I} \leftarrow \underline{I} + 1$   
5:  $\rightarrow \underline{2}$   
6: .
```

This definition defines an expansion of a for-statement into a piece of relocatable base language code. When the syntax of the class $[FS]$ is invoked dynamically the syntax analyzer is prepared to detect for-statements in the format specified in the above definition, and to substitute a block of base language text of equivalent meaning. For example, suppose the following for-statement were encountered in a piece of text written in a certain extension of the base language:

```
FOR  $I \leftarrow 1$  TO  $N$  DO  $S \leftarrow S + A[I]$ 
```

Since this is in the format specified in the above syntax definition, it would be recognized as an instance of the

syntactic class $[FS]$. Furthermore, the special variables $\underline{I}$, $\underline{L}$, $\underline{U}$ and $\underline{S}$ in the above definition, which we call extractor variables, would be assigned pieces of text as values. In particular, $\underline{I}$ would have the value "I", $\underline{L}$ would have the value "1", $\underline{U}$ would have the value "N" and $\underline{S}$ would have the value " $S \leftarrow S + A[\underline{I}]$ ". Here, the quote marks around an expression indicate that it is to be considered as a fragment of text and not as a rule for computing a value. After the extractor variables $\underline{I}$, $\underline{L}$, $\underline{U}$ and $\underline{S}$ have been assigned values, these values are substituted uniformly for the occurrences of $\underline{I}$, $\underline{L}$, $\underline{U}$ and $\underline{S}$ in the text specified in the body of the definition of $[FS]$. This act of substitution results in the creation of the following piece of relocatable text:

```

1:   $I \leftarrow 1$ 
2:   $(I > N) \rightarrow (\rightarrow \underline{NEXT})$ 
3:   $S \leftarrow S + A[\underline{I}]$ 
4:   $I \leftarrow I + 1$ 
5:   $\rightarrow \underline{2}$ 
6:  .

```

The syntax macro expansion process is applied iteratively to all the statements in this piece of text to expand instances of syntax macros if there are any. Then the text is bound.

That is, actual addresses are substituted for occurrences of the constructions $\rightarrow \underline{\text{NEXT}}$ and $\rightarrow \underline{2}$, depending on the location of the text relative to its substitution environment, and each statement in the text is renumbered to conform to the numbering of its substitution environment. A more complete example of the whole process is as follows: We consider the expansion of the following text, where S_1 and S_2 are assumed to be arbitrary base language statements.

```

4:  $S_1$ 
5: FOR  $I \leftarrow 1$  TO  $G$  DO FOR  $J \leftarrow 1$  TO  $H$  DO  $A[I, J] \leftarrow 0$ 
6:  $S_2$ 

```

Expansion of statement 5 yields the following list structure of relocatable text fragments (numbered in Dewey Decimal) :

```

4:  $S_1$ 
5: 5.1  $I \leftarrow 1$ 
   5.2  $(I > G) \rightarrow (\rightarrow \underline{\text{NEXT}})$ 
   5.3 5.3.1  $J \leftarrow 1$ 
       5.3.2  $(J > H) \rightarrow (\rightarrow \underline{\text{NEXT}})$ 
       5.3.3  $A[I, J] \leftarrow 0$ 
       5.3.4  $J \leftarrow J + 1$ 
       5.3.5  $\rightarrow \underline{2}$ 
   5.4  $I \leftarrow I + 1$ 
   5.5  $\rightarrow \underline{2}$ 
6:  $S_2$ 

```

Each consecutive line of the latter piece of text is then numbered (with a number in a circle). These circled numbers constitute the new line numbers to be used in the process of binding. The renumbered text appears as follows:

```

④ 4: S1
⑤ 5: ⑤ 5.1 I ← 1
      ⑥ 5.2 (I > G) → (→ NEXT)
      ⑦ 5.3 ⑦ 5.3.1 J ← 1
                ⑧ 5.3.2 (J > H) → (→ NEXT)
                ⑨ 5.3.3 A [I, J] ← 0
                ⑩ 5.3.4 J ← J+1
                ⑪ 5.3.5 → 2
      ⑫ 5.4 I ← I+1
      ⑬ 5.5 → 2
⑭ 6: S2

```

Binding is now carried out by replacing each occurrence of a relocatable address of the form N with the circled address to the left of the line N on its own level of the list structure, and by replacing each occurrence of NEXT by the circled number to the left of the line following the last line of level of the list structure containing the occurrence of NEXT .

In particular, this binding policy causes the following specific changes. (1) The statement $\rightarrow \underline{2}$ in line 5.3.5 is renumbered as $\rightarrow \underline{8}$ since statement 2 on the same level of the list structure, namely statement 5.3.2, has been renumbered as statement (8). (2) The statement $\rightarrow \underline{2}$ in line 5.5 is renumbered to be $\rightarrow \underline{6}$, since statement 2 on its same level, namely statement 5.2, has been renumbered as (6). (3) The statement $(J > H) \rightarrow (\rightarrow \underline{\text{NEXT}})$ is bound to be the statement $(J > H) \rightarrow (\rightarrow \underline{12})$ in line 5.3.2 since the next address at the next higher level than level 5.3.1 through 5.3.5 is the address associated with 5.4 which is (12). (4) Similarly, in line 5.2, the statement $(I > G) \rightarrow (\rightarrow \underline{\text{NEXT}})$ is renumbered to be the statement $(I > G) \rightarrow (\rightarrow \underline{14})$ since statement 6, renumbered as (14), is the next statement beyond the statements in the level 5.1 through 5.5. Thus the following text is the final result of the syntax macro expansion.

```

4: S1
5: I ← 1
6: (I > G) → (→ 14)
7: J ← 1
8: (J > H) → (→ 12)
9: A [I, J] ← 0
10: J ← J + 1
11: → 8
12: I ← I + 1
13: → 6
14: S2

```

This final text is obtained by creating a vector of text numbered with the new (or circled) numbers using statements whose addresses have been bound. Notice that this new piece of text is relocatable also.

Thus we see that the for-statement has been expanded into an equivalent section of text in the base language.

SYNTAX EXTENSION BY BROOKER AND MORRIS STYLE COMPILATION

In the following compiling feature of PPL, modeled after the Compiler-Compiler of Brooker and Morris [3,4,9] , the idea is to compose vectors of text. Each component of a text vector is a (base or extended) language statement. Expressions may be composed using variables having text as values. Arithmetic and Boolean operations cannot take place on text variables. Dotted operators prevent evaluation from occurring.

Syntactic constructions recognized as objects in the current syntax are passed as phrase structures (or, equivalently, parse trees or analysis records) to the routine associated with the syntax phrase definition. Here the semantics are defined by syntax macros, by Brooker and

Morris style compilation, or by a mixture of both.

In Brooker and Morris style compilation the phrase structures are broken down by pattern matching and extraction into text (or extractor) variables. These text variables are then used in expressions to create the vector of reduced text defining the semantics of the defined phrase. (Anyone familiar with the Brooker and Morris Compiler-Compiler [3,4,9] or with Formula Algol [10] will find the processes of compilation are based on the ideas of pattern matching, extraction of subexpressions and substitution found in those systems.)

An example of the compilation process will make things clear. Suppose we want to add a CASE statement to the base language such as the kind given in CPL [6] . Such statements are of the form:

SWITCH ON E INTO BEGIN S_1 ; S_2 ;...; S_n ; END

where each S_i for $1 \leq i \leq n$ is of the form $E :: \text{STATEMENT}$.

For example,

```

1: SWITCH ON X↑2 INTO ...
1:   BEGIN...
1:     25 :: X ← X-2 ;...
1:     ((Y-2)↑2) :: X ← X+5;...
1:     ALLOTHERS :: X ← ((Y-2)↑2);...
1:   END.
```

This will compile into the following piece of relocatable base language text:

```

1: j1 ← (X↑2) ⌈ [ 25, (Y-2)↑2 ]
2: → 1 + 2 × j1
3: X ← X-2
4: → NEXT
5: X ← X+5
6: → NEXT
7: X ← ((Y-2)↑2)
8: → NEXT

```

In statement 1 the iota operator of the form $Z \uparrow [V_1, V_2, \dots, V_i, \dots, V_n]$ has as value the least integer i for which $Z = V_i$ provided such V_i exists, otherwise the value is $n+1$. (This operator comes from Iverson's APL [2].)

The following syntax definitions define the syntax of case statements:

```

Δ [SWST] = [E] :: [E] ;
Δ [CASEST] = SWITCH ON C:[E] INTO BEGIN L:[*SWST] END
1: ↓ CMPL(C, L) .

```

The following routines define the compilation associated with case statements;

Δ CMPL = ∇ (C,L)

1: $V \leftarrow Q \leftarrow \text{NULLVECTOR}$

2: $(L = \underline{A} : [\text{SWST}]) \rightarrow (\rightarrow 1)$

3: $(L = \underline{A} : [\text{SWST}] \underline{B} : [* \text{SWST}]) \rightarrow (\rightarrow 4, \rightarrow 14)$

4: $L \leftarrow \underline{B}$

5: RESOLVE(A)

6: $\rightarrow 2$

7: RESOLVE(A)

8: $j \leftarrow \bar{j}$

9: $\text{TEXT} \leftarrow [j \leftarrow C \setminus V]$

10: $\text{TEXT} \leftarrow \text{TEXT} \cap [\rightarrow 1 + 2 \times j]$

11: FOR $I \leftarrow 1$ TO ρQ DO $\text{TEXT} \leftarrow \text{TEXT} \cap [Q[I], \rightarrow \underline{\text{NEXT}}]$

12: $\text{CMPL} \leftarrow \text{TEXT}$

13: $\rightarrow 0$

14: ("ERROR IN CASE STATEMENT")

Δ RESOLVE = ∇ (S)

1: $(S = (\underline{A} : [E]) :: \underline{B} : [E]) \rightarrow (\rightarrow 2, \rightarrow 5)$

2: $V \leftarrow (\underline{A} = \text{"ALLOTHERS"}) \rightarrow (V, V \cap \underline{A})$

3: $Q \leftarrow Q \cap \underline{B}$

4: $\rightarrow 0$

5: ("ERROR IN CASE", S) .

In the routine CMPL(C,L) , the actual parameters substituted for C and L respectively are the expression [E] on which switching will be performed and the list [*SWST] of switch cases. The CMPL routine breaks down the list of switch cases [*SWST] by concatenating the sequence of left hand sides of switch cases into a vector V and by concatenating the sequence of right hand sides of switch cases into the vector Q. It does this with the help of the auxiliary routine RESOLVE(S) whose parameter S is a [SWST]. In statement 1 of CMPL both V and Q are initialized to the null vector which is an identity under the concatenation operator $\cap$. In statements 2 and 3 the list L of switch cases is analyzed into its two possible formats : L consists of one and only one [SWST] (in statement 2) , and L consists of a [SWST] followed by [*SWST], where [*SWST] consists of one or more [SWST] 's . The extractor variables A and B receive as values the subexpressions ^{corresponding to} A[SWST] and [*SWST] respectively in case either pattern matches. If neither pattern matches, control is directed to statement 14 which prints an error. If $L == \underline{A}:[SWST] \underline{B}:[*SWST]$ then control is directed to statements 4,5 and 6 where L is assigned the remainder of the [*SWST] list, A is RESOLVED causing its ^{left hand side} ~~head~~ to be concatenated to V if it is not the expression "ALL OTHERS" and causing its ^{right hand side} ~~tail~~ to be concatenated to Q, and control is returned to statement 2 to complete the analysis of the remainder of the list of switch cases. If $L == \underline{A}:[SWST]$ control is

directed to statement 7 where A is RESOLVED and the construction of the vector of TEXT representing the result of ^{the} computation is constructed. This vector is constructed as follows. In statement 8, j is assigned a region choice $\bar{j}$. This region choice is a variable of the form "jN" where N is some integer not used in a previous evaluation of the region choice variable $\bar{j}$. (Briefly, region choice variables give us a mechanism for creating names.) The value of j is thus a piece of text of the form "jN". In statement 9 the first statement in the TEXT vector is created by evaluating the expression $[j \leftarrow C \{ V]$. Here, j is a variable name "jN", C is the ^{text of the} expression on which switching will be performed, and V is the ^{text of the} vector of left hand sides of cases constructed previously. Statement 10 concatenates to this first statement in TEXT a second statement of the form $\rightarrow 1 + 2 \times j$. Statement 11* concatenates to these first two statements a sequence of statement pairs of the form $[Q[I], \rightarrow \text{NEXT}]$, one for each statement in Q, or, in other words, one for each right hand side of each case in the case statement. In statement 12, the final vector of text is assigned to be the value of the variable CMPL which is the function designator of the procedure.

In the last line of page 24 the expression $\downarrow \text{CMPL}(\underline{C}, \underline{L})$ causes the execution of the procedure $\text{CMPL}(\underline{C}, \underline{L})$. The operator $\downarrow$ causes execution of its argument and is used in syntax macro definitions where normally substitution without execution is performed to create the expanded text.

* The operator ρ used in the expression ρQ , is Iverson's dimension operator which yields the length or dimension of Q. See [1,2].

The application of the syntax definition defined by the definitions given on the bottom of page 24 and its associated semantic compiling routines defined on page 25 expand the case statement given on the bottom of page 23 into the piece of base language text given on the top of page 24.

This concludes the section exemplifying the four methods of syntax extension.

SYSTEM FEATURES : VARIABLES, EXPRESSIONS, EDITING, LABELS, CONVERSTATION, STATE VECTORS, PRINTING, SUSPENSION, STOP CODES, TRACING, AND FILING

Typing an expression (special case: a variable) in conversational mode causes the expression to be evaluated and the value to be printed. The execution of functions may also be initiated in this manner. During execution a function may be suspended by pushing the interrupt (or attention) button on the terminal. A statement of the function name and line number to be executed next is typed out and conversational mode is entered. Execution may be resumed by executing the statement $\rightarrow N$ where N is the number of the line given at suspension. Alternatively, before resumption, intermediate values of variables and expressions may be requested or changed,

or definitions may be both created and edited. The state vector contains a record of all parallel processes currently being executed and a record of the nested function calls up to and including the point of suspension. The state vector may be printed out by requesting the value of the variable STATE. The state vector may also be initialized to empty by the command $STATE \leftarrow 0$. Any defined object in the system (syntax, functions, data definitions, etc) may be displayed and/or edited. The editing conventions follow APL [2], for the most part. Δ NAME reopens the definition of NAME and this definition may be altered. DISPLAY(NAME) displays this definition unless NAME is an operator, in which case, DISPLAYBINARY($\underline{\theta}$) displays the binary definition of $\underline{\theta}$ and DISPLAYUNARY($\underline{\theta}$) displays the unary definition of $\underline{\theta}$. Δ NAME[i] reopens the definition of NAME at line i provided the definition of NAME involves a set of $\geq i$ numbered lines. Typing a period closes the definition again. Δ NAME [0] allows the header of a definition to be altered and a fresh Δ will be typed on the next line to permit respecification of the header.

The command DELETE(NAME) , or in the cases of operators , the commands DELETEBINARY($\underline{\theta}$) and DELETEUNARY($\underline{\theta}$), delete any defined object or variable from the current state of the system. All variables are global except those specified as local to a function by a statement of the form LOCAL($X_1, \dots, X_n$).

There is no block structure in the base language , and all formal parameters to functions are called by value.

If any step of a function contains an expression E that is not an assignment, the value of E will be printed. If the symbol $\square$ is used as a subexpression of any expression on the right hand of an assignment, then when this $\square$ symbol is evaluated, control will return to the console, and a value will be requested. After this value has been typed, if a carriage return is executed the computation will resume with the typed value substituted for the occurrence of $\square$.

Arbitrary steps of a function may be traced or may have stop codes set on them by executing the respective commands: TRACE(NAME, (expression specifying a vector of steps to be traced)) and STOP(NAME, (expression specifying a vector of steps to be ^{stopped} ~~traced~~)). Both tracing codes and stop codes may be removed from a function NAME by using a null vector as the second argument in the latter two statements. These statements may be used as statements in programs implying that programs may be written to set and remove trace and stop codes under specified conditions.

To save the current state of the system as a file (all defined objects, the state vector, and all variables with their current values) do the command `SAVE(FILENAME)` or `SAVE(FILENAME:PASSWORD)`. To retrieve a file do `LOAD(FILENAME)` or `LOAD(FILENAME:PASSWORD)` if a password was used at the time of filing. To load a fresh working space into which files may be loaded incrementally, do the command `LOAD(NEWSPACE)`.

DYNAMIC SYNTAX AND SCOPE OF SYNTAX DEFINITIONS

Let $[E]$ denote the current syntax in effect at a given stage of extension of the base language. Suppose the definition of $[E]$ at this moment is $[E] = [E_1], [E_2], \dots, [E_n]$. If we do the command `INVOKE([X1], [X2], ..., [Xm])` then $[E]$ is expanded to have the form $[E] \leftarrow [E] \cup \{[X_1], [X_2], \dots, [X_m]\}$. In other words, $[E]$ is expanded to include those syntax phrases of the collection $\{[X_1], [X_2], \dots, [X_m]\}$ not already included in the alternates of $[E]$. Similarly, the command `REVOKE([X1], [X2], ..., [Xm])` has the effect of $[E] \leftarrow [E] - \{[X_1], [X_2], \dots, [X_m]\}$. Namely, any alternates of $[E]$ that are members of the collection $\{[X_1], [X_2], \dots, [X_m]\}$ are deleted from the current alternates of $[E]$.

If we think of $[E]$ as the definition of the current syntax in effect, then altering $[E]$ dynamically with the `INVOKE` and `REVOKE` commands causes dynamic changes in the

syntax extension of the base language currently in effect.

The INVOKE and REVOKE commands may be executed conversationally or as statements in programs.

We can take advantage of these commands by attempting to make a given syntax hold only for scopes where it is likely to be used. This can improve the efficiency of the PSG recognizer since it will be forced to try only "relevant" syntax alternatives. We can also use these commands to invoke different sets of definitions carrying different sets of semantics associated with a given notation. Thus a given syntax may be shared amongst two different meanings at two different times, (or N different meanings at M dynamically determined times).

PARALLEL TASKS AND MONITORING

One may define parallel tasks and monitors. For example:

```
Δ F = ∇ PARALLEL
1: WAITUNTIL(X=5)
2: SUSPEND(STATE)
3: F ← STATE[ρ STATE] .
```

This is a monitoring task that suspends the main subroutine stack STATE at the first time X=5 after it is executed, and control returns to the console with the name of the function and the number of its step at which suspension occurred.

Parallel tasks may suspend and call both regular routines and each other. E.g.

$\Delta G = \nabla \text{PARALLEL}$

1: WAITUNTIL(Y=2)

2: SUSPEND(F).

Here, when the routine G is active it will suspend F provided F is active soon after Y becomes 2. Another example is:

$\Delta H = \nabla \text{PARALLEL}(X)$

1: WAITUNTIL(Y=2)

2: F(X).

H(X) sets F(X) in motion in parallel then terminates itself soon after it detects that Y has become 2 for the first time.

When parallel routines call regular subroutines they contain

their own nest of executed subroutine calls.

The value of the variable TOTALSTATE is a vector of all state vectors of all parallel routines currently invoked together with the normal state vector STATE. TOTALSTATE may be set to empty by the command TOTALSTATE←0 .

CO-ROUTINES

Both regular routines and parallel routines may be made into co-routines by means of the RESUME commands of the form RESUME(F), RESUME(F [N]) and RESUME(F[N], X₁, X₂, ..., X_n) . These commands do the following in respective order ; (1) RESUME(F) resumes execution of the function F at the last point of suspension. (2) RESUME(F[N]) resumes F at step number N. (3) RESUME(F[N], X₁, X₂, ..., X_n) resumes F at step N with new values X₁, X₂, ... , X_n for the actual parameters of the routine F . The command SUSPEND(SELF) must be used when a routine G calls any parallel routine F in order to stop the execution of G following the call of F.

Examples of two parallel co-routines are given on the next page.

$\Delta A = \nabla \text{PARALLEL}(X,Y)$

1: $M \leftarrow X \uparrow 2$

2: $B(M,Y)$

3: $\text{SUSPEND}(\text{SELF})$

4: $K \leftarrow Y \uparrow 2$

5: $\text{RESUME}(B, M,K)$

6: $\text{SUSPEND}(\text{SELF})$

7: $[X,Y]$.

$\Delta B = \nabla \text{PARALLEL}(U,V)$

1: $M \leftarrow U \uparrow (1 \div 2)$

2: $\text{RESUME}(A[4], M,V)$

3: $\text{SUSPEND}(\text{SELF})$

4: $K \leftarrow V \uparrow (1 \div 2)$

5: $\text{RESUME}(A)$.

These two routines alternately call each other in a conversation in which whatever A does, B undoes. The command $B(M,Y)$ at $A[2]$ sets routine B in motion in parallel (since B was defined as parallel) with the concurrent execution of A. While B starts to work, A executes statement $A[3]$ in which it suspends itself and goes to sleep. The command $B[2]$ resumes the execution of A at step 4 with new actual parameters M and V, then suspends itself. Then A resumes B at step $A[5]$, whence B resumes execution at step $B[4]$. Finally B resumes A at $A[7]$ whence the values of X and Y are printed.

REFERENCES

The concept of the reference allows the construction of

richly connected data structures. The reference operator " $\rightarrow$ " is used to create references. It takes the left hand value (or reference to) its argument as a value. (The terminology is Strachey's from CPL [6]). This value is of type REFERENCE and prints as an unsigned integer signifying an address in the machine. (These addresses may be different for different executions of a given program.) E.g.

$$\Delta Y = \text{REFERENCE} \vee \text{INTEGER}$$

$$Y \leftarrow \overset{\text{PAIR}}{\text{LIST}}(\text{MATRIX}(2,3,Y)) \cup (1,2,3,4,5,6), \text{NULLLIST})$$

$$X \leftarrow \rightarrow \text{HEAD}(Y)[1,2]$$

$$Z \leftarrow \rightarrow \rightarrow \text{HEAD}(Y)[1,2]$$

$$Z \leftarrow \rightarrow X$$

Notes: In normal assignment $X \leftarrow Y$, X receives a copy of Y. Whenever, an expression E whose value is type reference is used on the left hand of an assignment, the left hand value used in the assignment is that supplied by the reference value. In the above example X contains a value of type REFERENCE which gives the address of the [1,2] element of the Head element of the list Y , and Z contains the address of the cell containing the address of the [1,2] element of the Head of the list Y .

The inverse of the reference operator is the contents operator written $\subset$. We observe the identity that $\subset(\rightarrow X) = X$.

Arbitrarily many levels of indirect addressing can be created with chains of reference operators of the form

→ . . . → . Reference values behave arithmetically like unsigned integers.

INPUT-OUTPUT IN PPL

Input-Output is machine dependent. However, if we are given arrays that function as I/O character buffers and some basic I/O commands (e.g. upspace printer or carriage return on teletype) we can build up elegant format languages by extension.

THE COMPILE OPERATOR

There is a compile operator COMPILE(F) which when applied to the name of a function "F" maps its text into machine code. When a compiled function is edited it must be recompiled.

Type declarations for variables such as DECLARE(REAL,X,Y) make compiled code more efficient by reducing the amount of interpretation and selection.

APPENDIX I

The Precedence Matrix of the Grammar G is as follows:

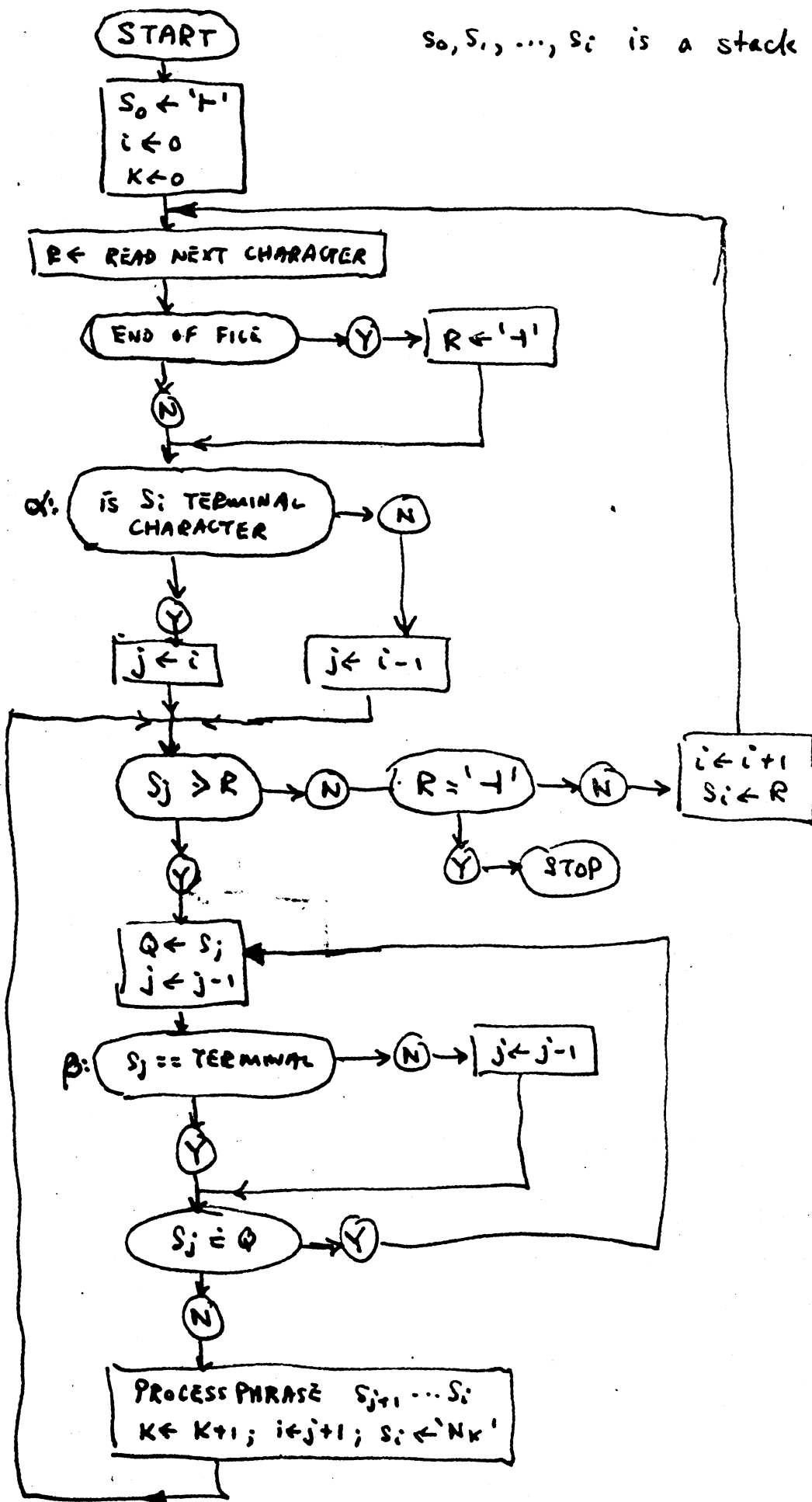
	(	[	λ	)	]	α	β	,
)	$\succ$	$\succ$		$\succ$	$\succ$		$\succ$	$\succ$
]	$\succ$	$\succ$		$\succ$	$\succ$		$\succ$	$\succ$
λ	$\succ$	$\succ$		$\succ$	$\succ$		$\succ$	$\succ$
(	$\prec$	$\prec$	$\prec$	$\doteq$		$\prec$	$\prec$	$\prec$
[	$\prec$	$\prec$	$\prec$		$\doteq$	$\prec$	$\prec$	$\prec$
α	$\prec$	$\prec$	$\prec$	$\succ$	$\succ$	$\prec$	$\prec$	$\succ$
β	$\prec$	$\prec$	$\prec$	$\succ$	$\succ$	$\prec$	$\prec$	$\succ$
,	$\prec$	$\prec$	$\prec$	$\succ$	$\succ$	$\prec$	$\prec$	$\prec$

The Precedence Functions for the Grammar G are as Follows:

	F(x)	G(x)
)	6	1
]	6	1
λ	6	5
(	1	5
[	1	5
α	4	5
β	4	5
,	2	3
$\vdash$	0	7
$\dashv$	7	0

where we have:

$$\begin{aligned}
 \psi < \phi & \text{ iff } F(\psi) < G(\phi) \\
 \psi & \doteq \phi & \text{ iff } F(\psi) = G(\phi) \\
 \psi > \phi & \text{ iff } F(\psi) > G(\phi)
 \end{aligned}$$



Floyd's Precedence Algorithm :

-40-

APPENDIX II
Implementation Notes for PPL

1. A modified form of Earley's algorithm has been discovered which enables the Brooker and Morris format to be used to describe grammatical phrase classes. The Brooker and Morris formalism is an extension of Backus Normal Form in which operators for optional presence and indefinite repetition of syntactic constructions have been added (namely the * and ? operators). An additional closure operation is needed to implement the optional presence operator. Earley's algorithm, extended in this fashion, is used to recognize all sentences in extensions of the base language. The precedence algorithm, given in Appendix I, is used to recognize all sentences in the base language.

2. The lexical analyzer has three parts. A tabular representation of a finite automaton, following a method given by D.L.Parnas, is used to recognize identifiers, constants, operators and reserved words. The identifiers and reserved words are hashed into associated internal forms. The finite automaton switches off to coding and hashing routines which map character strings into internal form. Both the tabular representation of the finite automaton and the coding routines that deal with the translation of lexemes into internal form can be edited. Thus the programmer has a measure of control over the nature of the lexical analysis performed in PPL. Finally, for multiple operators (such as == and .+), a discrimination net is used as a memory device. This net changes dynamically as multiple operators are added and subtracted from the syntax during dynamic syntax variation.

3. Functions in the system are represented as pure procedures. A given execution of a function causes the creation of an activation record which contains all information relevant to that execution. For example, the control counter, all local variables, the value of the procedure identifier,

the current status (such as suspended or active) and so forth, are all kept in the activation record of a given function. This enables the implementation of many different control structures such as parallel routines and co-routines since many procedures may each share the same pure procedure instruction set, each in a different stage of execution. A time-sharing executive simulates the parallel activity of many procedures simultaneously active.

4. The implementation of monitoring tasks is implemented through the device of continuously evaluating expressions invented by D. Fisher.

5. For the implementation of the reference operator $\rightarrow X$, the following device is used. Right hand values are always obtained by applying a contents operator to left hand values. Before this contents operator is applied, the left hand value is saved in a cell. Thus the right hand and left hand values of a given operand are always available. The right hand values are typically used in the construction of new values. The left hand values are typically used on the left hand sides of assignments and as arguments of the reference operator. The action of a reference operator on its operand, therefore, is to construct a new value of type REFERENCE using the left hand value of the operand.

6. There are, of course, various classes of primitive functions in the language for doing arithmetic and logic on internal representations of numerical and logical operands. There are also primitive constructors, predicates and selectors used as a basis for the interpretation of data definitions.

REFERENCES

- 1 Iverson, K.E., and Falkoff, A.D., APL/360 MANUAL OF OPERATION, IBM Corp., Yorktown Heights, Nov. 1967
- 2 Iverson, K.E., A Programming Language, New York, Wiley, 1962
- 3 Brooker, R.A. and Morris, D., A General Translator Program for Phrase Structure Languages, JACM, vol.9, No.1, Jan. 1962, p.1
- 4 Brooker, R.A., Morris, D. and Rohl, J.S., Experience with the Compiler-Compiler, The Computer Journal, vol.9, No.4, Feb. 1967, pp. 345-349
- 5 Standish, T.A., A Data Definition Facility for Programming Languages, doctoral dissertation, Carnegie Institute of Technology, May 1967
- 6 Strachey, C., Barron, D.W., Hartley, D.F., Buxton, J.N. and Nixon, E. CPL Working Papers, University of London Institute of Computer Science and the Mathematical Laboratory, Cambridge, July 1966
- 7 Church, A., The Calculi of Lambda Conversion, Princeton University Press, Princeton, 1941
- 8 Floyd, R.W., Syntactic Analysis and Operator Precedence, JACM, vol. 10, July 1963, pp. 316-333
- 9 Rosen, S. A Compiler Building System Developed by Brooker and Morris, CACM, vol.7, July 1964, pp. 403-414
- 10 Perlis, A.J., Iturriaga, R., and Standish, T.A., A Definition of Formula Algol, Department of Computer Science, Carnegie Institute of Technology, March 1966
- 11 McCarthy, John, et al., Lisp 1.5 Programmer's Manual, MIT Press, Cambridge, Massachusetts, (1962)